ECOLE MAROCAINE DES SCIENCES DE L'INGENIEUR
Membre de HONORIS UNITED UNIVERSITIES

PROJET DE FIN D'ANNÉE

5ème Année en Ingénierie Informatique et Réseaux

FlightRadar
Système de Suivi de Vol en Temps Réel

Réalisé par :
Saad Chatiou
Abdeladim Jouhlali
Salaheddine Sahnoune

Tuteur(s) :
Encadrant académique : Mr Chawki EL Balmany

Organisme d'accueil :
Emsi

Année universitaire : 2025/2026

Dédicaces

À mes parents, pour leur amour inconditionnel, leur soutien sans faille et les nombreux sacrifices qu'ils ont consentis pour me permettre de poursuivre mes études dans les meilleures conditions.

À ma famille pour leur encouragement constant tout au long de ce parcours.

À toutes les personnes qui, de près ou de loin, ont contribué à la réalisation de ce projet.

Remerciements

Ce projet n'aurait jamais pu voir le jour sans l'aide et le soutien de plusieurs personnes.

Je tiens tout d'abord à exprimer ma profonde gratitude à mon encadrant académique, pour ses conseils avisés et son accompagnement durant tout le déroulement de ce projet.

Table des matières

Dédicaces 1
Remerciements 2

Liste des figures 6

1 Introduction Générale 7
1.1 Contexte Général 7
1.2 Problématiques 7
1.2.1 Problèmes Techniques 7
1.2.2 Problèmes Fonctionnels 7
1.3 Objectifs 8
1.3.1 Objectif principal 8
1.3.2 Objectifs techniques 8
1.3.3 Objectifs fonctionnels 8
1.4 Structure du Rapport 8

2 État de l'Art 9
2.1 Background sur les Technologies 9
2.1.1 Framework Frontend : React 9
2.1.2 Framework Backend : Spring Boot 9
2.1.3 Base de Données : PostgreSQL 9
2.1.4 Messagerie : Apache Kafka 10
2.1.5 Communication Temps Réel : WebSocket (STOMP) 10
2.1.6 Cartographie : Leaflet 10
2.2 Benchmarking des Technologies 11
2.2.1 Comparaison Frontend : React vs Vue vs Angular 11
2.2.2 Comparaison Backend : Spring Boot vs Node.js vs Django . . . 11
2.2.3 Comparaison Base de Données : PostgreSQL vs MySQL vs MongoDB 11

3 Conception et Modélisation 12
3.1 Les Diagrammes 13
3.1.1 Diagramme de Cas d'Usage (Use Case) 13
3.1.2 Diagramme de Séquence : Création de Vol 15
3.1.3 Diagramme de Classes 16
3.1.4 Diagramme BPMN : Cycle de Vie d'un Vol 19

4 Architecture Proposée 21
4.1 Vue d'Ensemble de l'Architecture 21
4.2 Architecture Frontend 23

4.2.1 Structure des Composants 23
4.2.2 Flux de Communication Frontend-Backend 23
4.3 Architecture Backend 24
4.3.1 Structure des Packages 24
4.3.2 Services Principaux 24
4.4 Architecture de la Base de Données 25
4.4.1 Schéma Relationnel 25
4.4.2 Triggers SQL 26
4.5 Architecture Kafka 26
4.5.1 Topics Kafka 26
4.6 Architecture WebSocket 27
4.6.1 Configuration STOMP 27
4.6.2 Topics WebSocket 27
4.6.3 Flux de Communication 27
4.7 Technologies et Versions 28
4.7.1 Frontend 28
4.7.2 Backend 28

5 Implémentation 29
5.1 Captures d'écran 29
5.1.1 Page de connexion 29
5.1.2 Cockpit Administrateur - Gestion des vols 30
5.1.3 Dashboard Pilote - Radar de vol 31
5.1.4 Dashboard Pilote - Statut du vol 31
5.1.5 Dashboard Pilote - Aucun vol actif 32
5.1.6 Radar - Carte des avions 33
5.1.7 Radar - Positions et alertes 33
5.1.8 Cockpit Admin - Création de vol 34
5.2 Code Source Important 35
5.2.1 Service de Gestion de Vol 35
5.2.2 Service de Détection de Conflits 36
5.2.3 Hook WebSocket Frontend 37
5.2.4 Tableau récapitulatif des captures 39

Conclusion Générale 40
.1 Extraits de code source complets 42
1.1 Configuration Spring Security (SecurityConfig.java) 42
1.2 Configuration Kafka (KafkaConfig.java) 44
1.3 Hook WebSocket React (useWebSocket.js) 46
.2 Scripts SQL 48
2.1 Schéma de base de données complet 48
2.2 Séquences PostgreSQL pour génération de numéros de vol 51
.3 Configuration Docker Compose 51
3.1 docker-compose.yml (Architecture proposée) 51
.4 Configuration Frontend (package.json) 53
4.1 Dépendances principales 53
.5 Exemples de messages Kafka 54
5.1 Événement de création de vol 54
5.2 Message de télémétrie d'avion 54

5.3 Message VHF....55
.6 Format des réponses API REST....55
6.1 Réponse de connexion (POST /api/auth/login)....55
6.2 Réponse du dashboard radar (GET /api/radar/dashboard)....56

Table des figures

3.1 Diagramme de cas d'usage - Partie 1/2 13
3.2 Diagramme de cas d'usage - Partie 2/2 14
3.3 Diagramme de séquence - Création d'un vol 15
3.4 Diagramme de classes du système 16
3.5 Diagramme BPMN - Cycle de vie d'un vol 19

4.1 Architecture globale du système FlightRadarPFA 22
4.2 Relations entre tables principales 25

5.1 Interface de connexion au système 29

Chapitre 1

Introduction Générale

1.1 Contexte Général

Le secteur aéronautique nécessite des systèmes de contrôle du trafic aérien fiables pour gérer le trafic croissant. FlightRadarPFA est un système de suivi de vol en temps réel qui permet :

— La visualisation des positions d'avions sur une carte interactive

— La gestion des vols et des communications VHF

— La détection de conflits entre avions

— La surveillance météorologique

— La coordination entre pilotes et centres radar

1.2 Problématiques

1.2.1 Problèmes Techniques

1. Intégration de données ADS-B en temps réel : Collecte et traitement des données de position depuis différentes sources (OpenSky Network, feeders ADS-B)
2. Communication temps réel : Mises à jour instantanées des positions pour les pilotes et contrôleurs aériens via WebSocket
3. Détection de conflits : Algorithme de calcul du CPA (Closest Point of Approach) pour prévenir les collisions potentielles
4. Gestion des communications VHF : Transcription audio et gestion des messages entre avions et centres radar
5. Scalabilité : Architecture capable de gérer des milliers d'avions simultanément

1.2.2 Problèmes Fonctionnels

— Synchronisation entre différentes vues

— Validation métier (1 pilote = 1 avion EN_VOL maximum)

— Gestion des statuts de vol (EN COURS, TERMINE)

— Stockage de l'historique des vols

1.3 Objectifs

1.3.1 Objectif principal

Développer un système complet de suivi de vol en temps réel avec architecture modulaire.

1.3.2 Objectifs techniques

— Architecture monolithique frontend/backend

— Communication temps réel via WebSocket(STOMP)

— Messagerie asynchrone avec Apache Kafka

— API REST sécurisée avec JWT

— Base de données PostgreSQL avec triggers SQL

1.3.3 Objectifs fonctionnels

— Dashboard pilote

— Dashboard radar

— Communication VHF avec transcription audio

— Détection automatique de conflits

— Intégration météorologique

1.4 Structure du Rapport

— Chapitre 1 : Présente l'état de l'art des technologies utilisées (React, Spring Boot, PostgreSQL, Kafka, WebSocket)

— Chapitre 2 : Décrit la conception et la modélisation avec les diagrammes UML (use case, séquence, classe) et BPMN

— Chapitre 3 : Détaille l'architecture proposée avec les schémas techniques (API, Frontend, Backend, Base de données)

— Chapitre 4 : Présente l'implémentation avec des captures d'écran et du code source

— Conclusion : Synthétise les résultats et les perspectives d'évolution

Chapitre 2

État de l'Art

2.1 Background sur les Technologies

2.1.1 Framework Frontend : React

React est une bibliothèque JavaScript open-source développée par Facebook pour construire des interfaces utilisateur. Version utilisée : React 18.2.0.

Caractéristiques principales :

— Composants réutilisables

— Virtual DOM pour performance

— Hooks pour la gestion d'état

— Écosystème riche (React Router, React-Leaflet)

Choix pour FlightRadarPFA : React permet de créer des interfaces réactives avec mises à jour temps réel, idéal pour les dashboards de suivi de vol.

2.1.2 Framework Backend : Spring Boot

Spring Boot est un framework Java qui simplifie le développement d'applications. Version utilisée : Spring Boot 3.2.0 avec Java 17.

Caractéristiques :

— Auto-configuration

— Architecture modulaire (Controllers, Services, Repositories)

— Intégration native avec JPA/Hibernate

— Support WebSocket intégré

Choix pour FlightRadarPFA : Spring Boot offre une base solide pour une API REST sécurisée avec gestion des transactions et intégration de services.

2.1.3 Base de Données : PostgreSQL

PostgreSQL est un SGBD relationnel open-source, robuste et performant.

Caractéristiques :

— Support des triggers SQL

— Fonctions stockées

— Transactions ACID

— Performances élevées pour données relationnelles

Choix pour FlightRadarPFA : PostgreSQL permet de gérer les données structurées (vols, avions, pilotes) avec des contraintes d'intégrité et des triggers pour automatiser certaines opérations.

2.1.4 Messagerie : Apache Kafka

Apache Kafka est une plateforme de streaming distribuée pour la gestion de flux de données en temps réel.

Caractéristiques :

— Découplage des services

— Persistance des messages

— Scalabilité horizontale

— Consumer groups pour traitement parallèle

Choix pour FlightRadarPFA : Kafka permet de découpler la production et la consommation d'événements (création de vol, télémétrie, messages VHF), assurant fiabilité et performance.

2.1.5 Communication Temps Réel : WebSocket (STOMP)

WebSocket est un protocole de communication bidirectionnel. STOMP (Simple Text Oriented Messaging Protocol) est un protocole de messagerie sur WebSocket.

Caractéristiques :

— Communication bidirectionnelle

— Faible latence

— Support des topics et queues

— Reconnexion automatique

Choix pour FlightRadarPFA : WebSocket permet d'envoyer des mises à jour de position en temps réel aux clients sans polling, réduisant la charge serveur et améliorant l'expérience utilisateur.

2.1.6 Cartographie : Leaflet

Leaflet est une bibliothèque JavaScript open-source pour cartes interactives.

Caractéristiques :

— Légère et performante

— Intégration avec OpenStreetMap

— Support des marqueurs et polylines

— Personnalisation des icônes

Choix pour FlightRadarPFA : Leaflet permet d'afficher les avions sur une carte avec mise à jour en temps réel des positions.

2.2 Benchmarking des Technologies

2.2.1 Comparaison Frontend : React vs Vue vs Angular

Critère	React	Vue	Angular
Courbe d'apprentissage	Moyenne	Facile	Difficile
Performance	Excellente	Excellente	Bonne
Écosystème	Très large	Large	Très large
Taille bundle	Petite	Petite	Grande

TABLE 2.1 – Comparaison des frameworks frontend

Choix : React pour son écosystème et sa performance.

2.2.2 Comparaison Backend : Spring Boot vs Node.js vs Django

Critère	Spring Boot	Node.js	Django
Performance	Excellente	Excellente	Bonne
Gestion transactions	Native	Manuel	ORM
WebSocket	Intégré	Native	Channels
Sécurité	Spring Security	Manuel	Django Auth

TABLE 2.2 – Comparaison des frameworks backend

Choix : Spring Boot pour la gestion des transactions et la sécurité.

2.2.3 Comparaison Base de Données : PostgreSQL vs MySQL vs MongoDB

Critère	PostgreSQL	MySQL	MongoDB
Transactions ACID	Oui	Oui	Limitée
Triggers SQL	Oui	Oui	Non
Requêtes complexes	Excellentes	Bonnes	Limitées
Scalabilité	Verticale	Verticale	Horizontale

TABLE 2.3 – Comparaison des bases de données

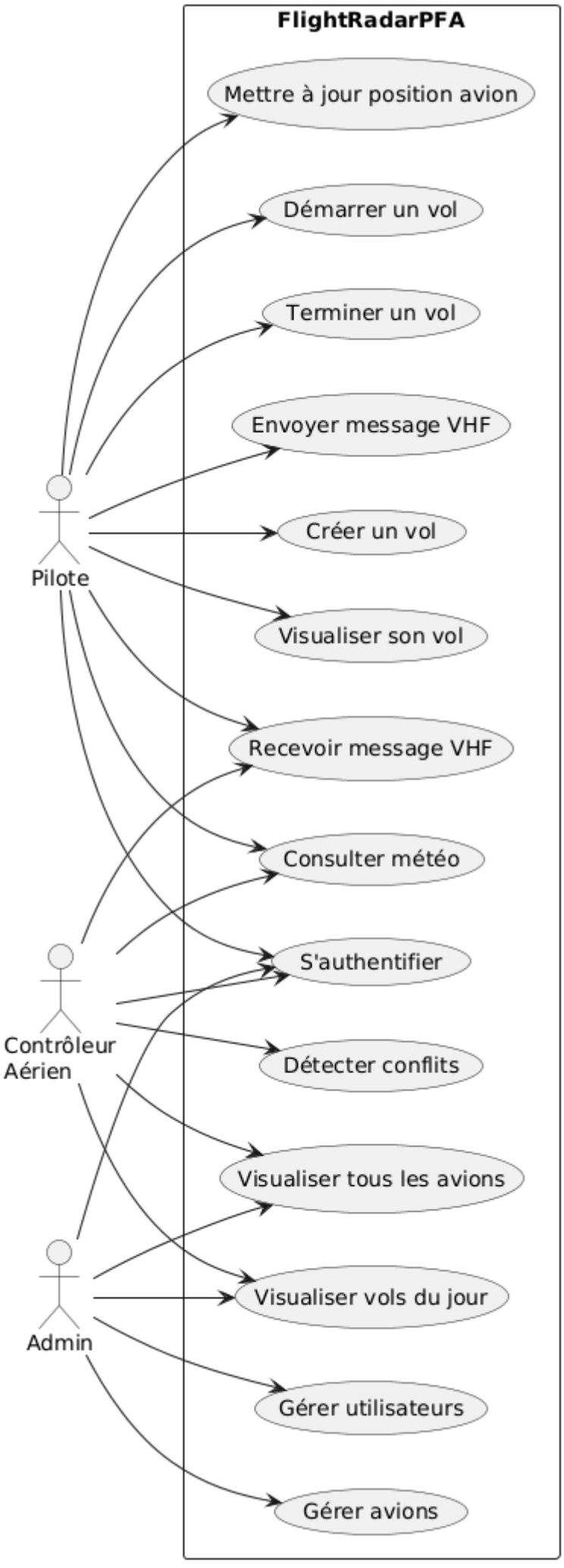
Choix : PostgreSQL pour les triggers SQL et les requêtes complexes.

Chapitre 3

Conception et Modélisation

3.1 Les Diagrammes

3.1.1 Diagramme de Cas d'Usage (Use Case)


FlightRadarPFA
Mettre à jour position avion
Démarrer un vol
Terminer un vol
Envoyer message VHF
Créer un vol
Visualiser son vol
Recevoir message VHF
Consulter météo
S'authentifier
Détecter conflits
Visualiser tous les avions
Visualiser vols du jour
Gérer utilisateurs
Gérer avions
Pilote
Contrôleur Aérien
Admin

Description : Ce diagramme présente les acteurs principaux du système FlightRadarPFA et leurs interactions avec les différentes fonctionnalités.

Acteurs identifiés :

— **Pilote** : Peut créer et gérer ses vols, consulter sa position en temps réel, communiquer via VHF avec les contrôleurs, et recevoir des alertes.

— **Contrôleur Aérien** : Surveille le trafic aérien, détecte les conflits potentiels, gère les communications VHF, et valide les plans de vol.

— **Administrateur** : Gère les utilisateurs, les avions, les aéroports, et consulte les statistiques du système.

Fonctionnalités principales :

— Authentification et gestion des rôles

— Création et suivi des vols

— Visualisation cartographique en temps réel

— Communication VHF bidirectionnelle

— Détection automatique des conflits

— Génération de rapports et statistiques

3.1.2 Diagramme de Séquence : Création de Vol

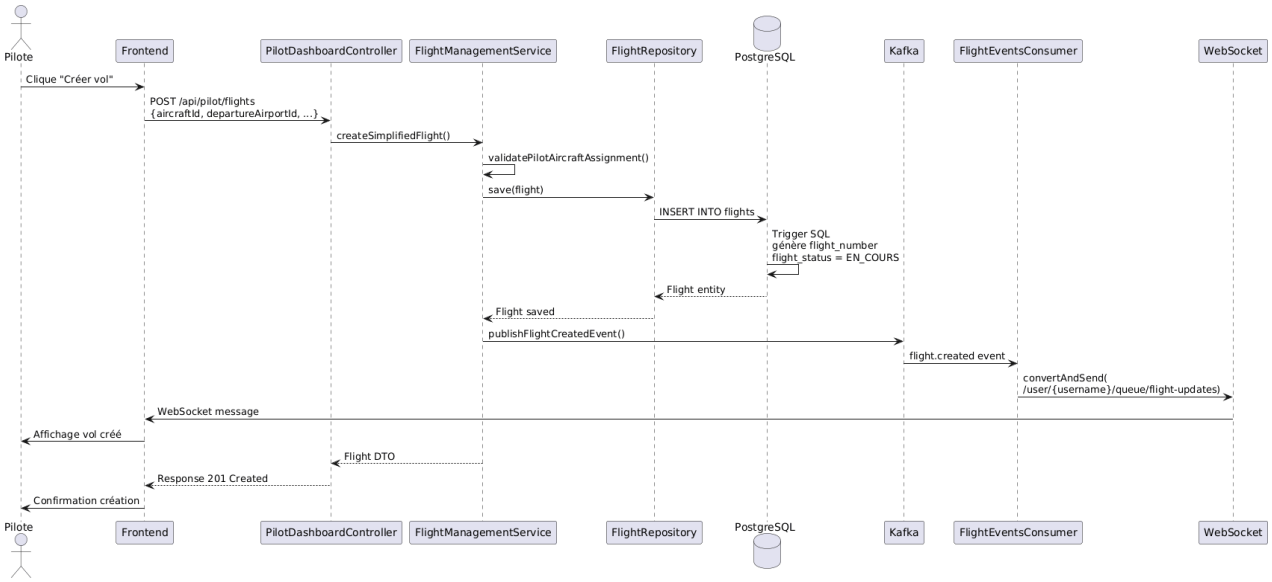


FIGURE 3.3 – Diagramme de séquence - Création d'un vol

Description : Ce diagramme illustre le flux séquentiel des interactions entre les différents composants lors de la création d'un vol.

Flux détaillé :

1. **Soumission du formulaire** : Le pilote remplit et soumet le formulaire de création de vol via l'interface React.
2. **Appel API REST** : Le frontend envoie une requête POST à l'endpoint `/api/flights` avec les données JSON du vol.
3. **Validation backend** : Le contrôleur Spring Boot valide les données d'entrée (champs obligatoires, formats).

4. **Vérification métier** : Le service vérifie la disponibilité de l'avion et s'assure que le pilote n'a pas déjà un vol en cours.
5. **Persistance des données** : Insertion en base PostgreSQL avec déclenchement d'un trigger SQL pour génération automatique du numéro de vol.
6. **Publication événement** : Publication d'un événement `flight.created` sur le topic Kafka dédié.
7. **Notification temps réel** : Diffusion via WebSocket aux contrôleurs concernés pour mise à jour instantanée.
8. **Confirmation utilisateur** : Affichage d'un message de succès au pilote dans l'interface React.

3.1.3 Diagramme de Classes

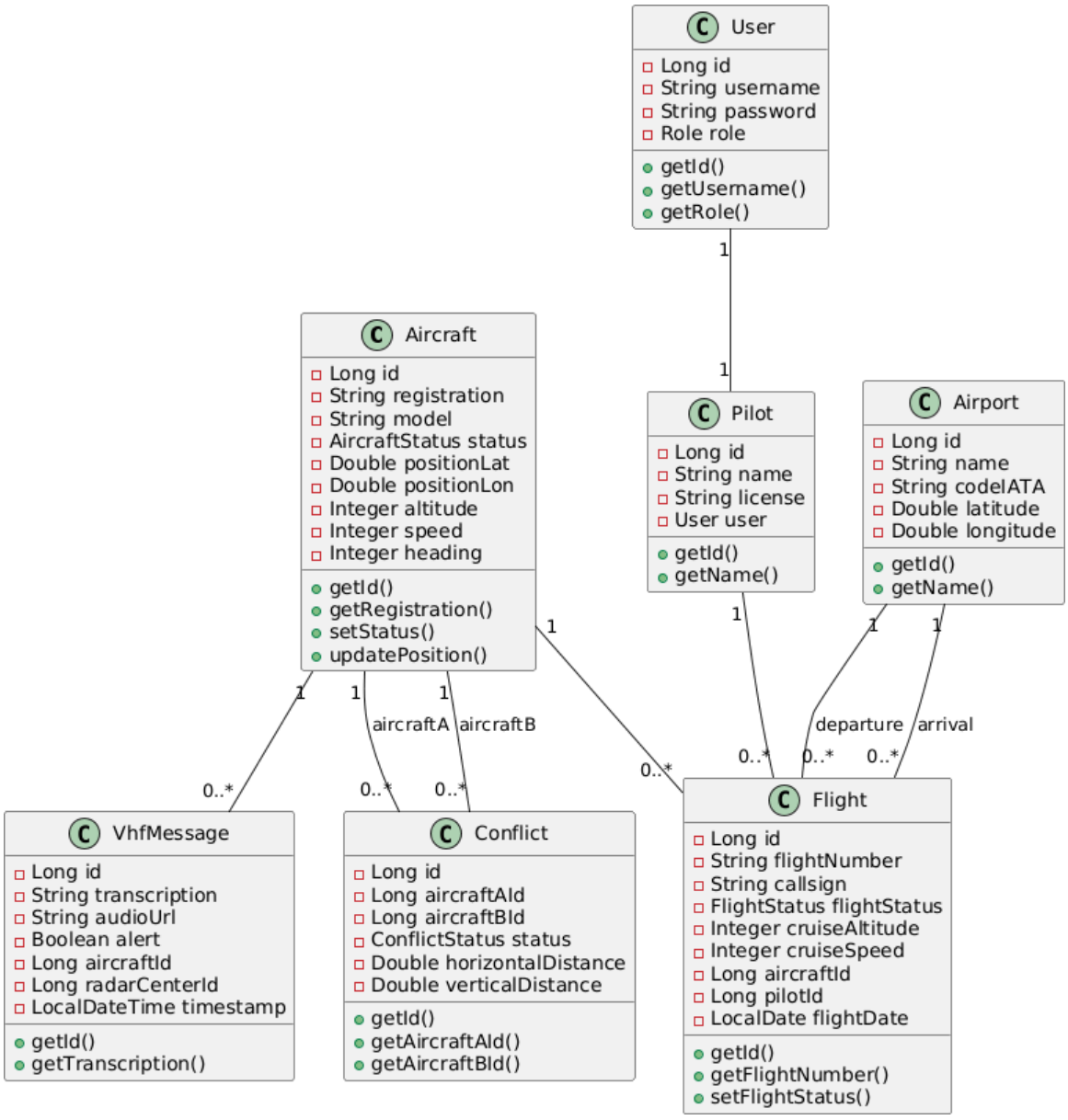


FIGURE 3.4 – Diagramme de classes du système

Description : Modèle conceptuel des entités principales du système et de leurs relations.

Classes principales :

— **User** : Classe abstraite représentant un utilisateur avec authentification JWT.

— **Pilot** : Spécialisation de User, avec des attributs spécifiques (licence, heures de vol).

— **Aircraft** : Représente un aéronef avec ses caractéristiques techniques et son état opérationnel.

— **Flight** : Modélise un vol avec ses informations de planification et d'exécution.

— **Airport** : Représente un aéroport avec ses caractéristiques (pistes, coordonnées).

— **VhfMessage** : Gère les communications vocales entre avions et centres de contrôle.

— **Conflict** : Enregistre les conflits détectés entre aéronefs.

Relations cardinales :

— **Aircraft ↔ Flight** : Relation 1-N (un avion peut effectuer plusieurs vols, mais un vol concerne un seul avion).

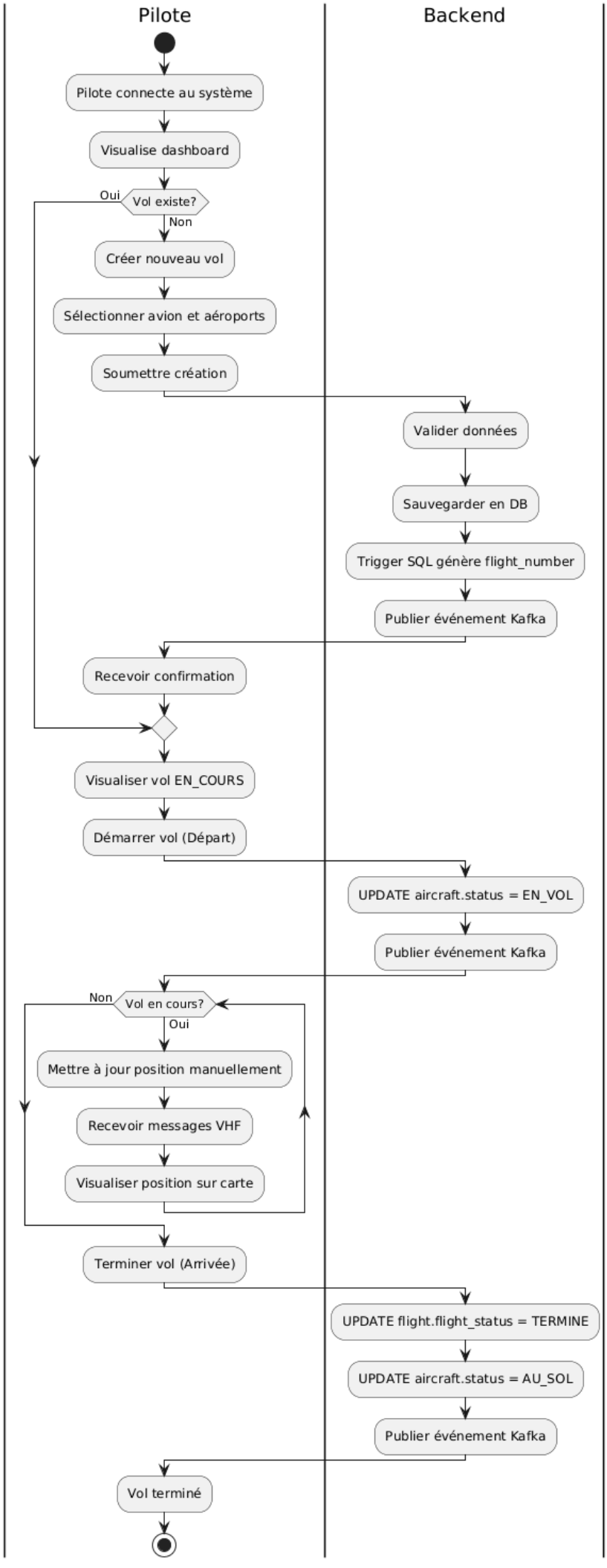
— **Pilot ↔ Flight** : Relation 1-N (un pilote peut effectuer plusieurs vols, mais un vol a un seul pilote).

— **Airport ↔ Flight** : Relation 2-1 (un vol a exactement un aéroport de départ et un d'arrivée).

— **VhfMessage ↔ Aircraft** : Relation N-N (un avion peut échanger plusieurs mes-

— sages).

3.1.4 Diagramme BPMN : Cycle de Vie d'un Vol


Pilote
Backend
Pilote connecte au système
Visualise dashboard
Oui
Vol existe?
Non
Créer nouveau vol
Sélectionner avion et aéroports
Soumettre création
Valider données
Sauvegarder en DB
Trigger SQL génère flight_number
Publier événement Kafka
Recevoir confirmation
Visualiser vol EN_COURS
Démarrer vol (Départ)
UPDATE aircraft.status = EN_VOL
Publier événement Kafka
Non
Vol en cours?
Oui
Mettre à jour position manuellement
Recevoir messages VHF
Visualiser position sur carte
Terminer vol (Arrivée)
UPDATE flight.flight_status = TERMINE
UPDATE aircraft.status = AU_SOL
Publier événement Kafka
Vol terminé

Description : Modélisation du processus métier complet de gestion d'un vol, de sa planification à son achèvement.

Phases du cycle :

1. **Planification** :
   - Définition des paramètres du vol (aéroports, horaires, altitude)
   - Sélection de l'avion et du pilote
   - Validation préliminaire des contraintes
2. **Validation** :
   - Vérification automatique des règles de sécurité
   - Contrôle de la disponibilité des ressources
   - Approbation par le système ou le contrôleur
3. **Décollage** :
   - Mise à jour du statut de l'avion
   - Début de l'enregistrement de la télémétrie
   - Notification aux parties prenantes
4. **En vol** :
   - Surveillance continue via ADS-B
   - Communication VHF avec les centres
   - Détection automatique des conflits
   - Mises à jour en temps réel
5. **Atterrissage** :
   - Procédure d'approche et d'atterrissage
   - Mise à jour du statut
   - Arrêt de l'enregistrement
6. **Clôture** :
   - Génération du rapport de vol
   - Archivage des données
   - Libération des ressources
   - Mise à jour des statistiques

Chapitre 4

Architecture Proposée

4.1 Vue d'Ensemble de l'Architecture

L'architecture de FlightRadarPFA suit une approche monolithique modulaire avec séparation claire entre frontend et backend.

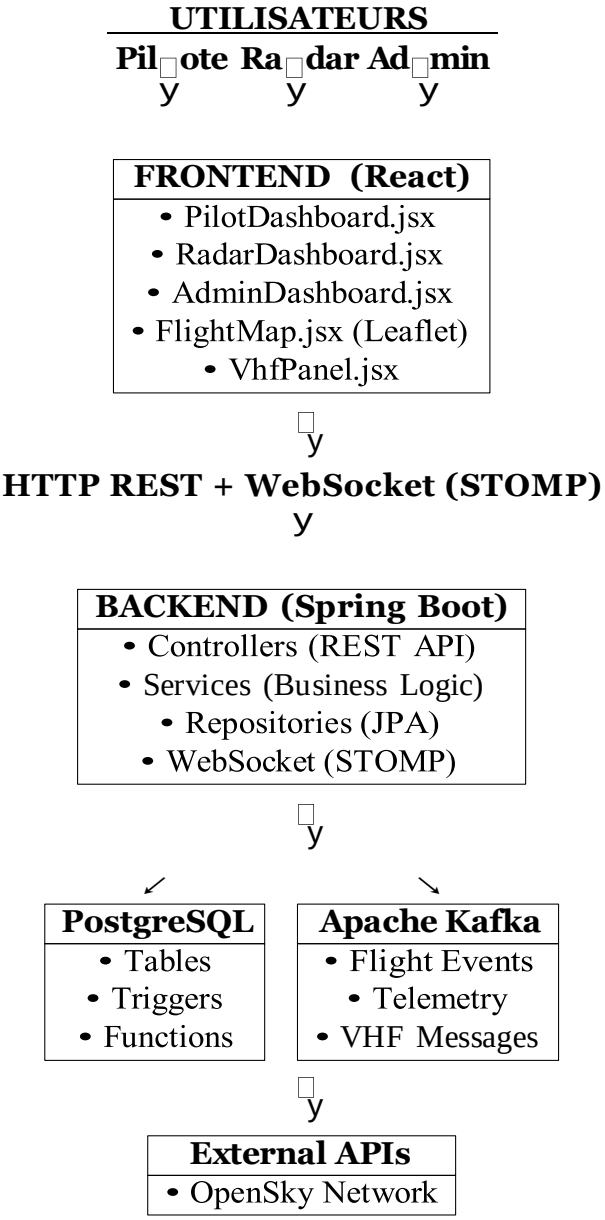


FIGURE 4.1 – Architecture globale du système FlightRadarPFA

4.2 Architecture Frontend

4.2.1 Structure des Composants

Dossier/Fichier	Description
src/main.jsx	Point d'entrée de l'application
src/App.jsx	Router principal + routes protégées
src/pages/	• PilotDashboard.jsx → Dashboard pilote • RadarDashboard.jsx → Dashboard radar • AdminDashboard.jsx → Dashboard admin
src/components/	• FlightMap.jsx → Carte Leaflet • VhfPanel.jsx → Communications VHF • VhfSender.jsx → Envoi messages • MetarWeatherPanel.jsx → Météo
src/services/	• api.js → Client Axios • websocket.js → Client WebSocket
src/hooks/	• useWebSocket.js → Hook WebSocket • useVhfConflict.js → Détection conflits
src/context/	AuthContext.jsx → Gestion authentification

TABLE 4.1 – Structure des fichiers frontend

4.2.2 Flux de Communication Frontend-Backend

REST API	
Base URL	http://localhost:8080/api
Authentification	JWT dans header : Authorization: Bearer {token}
Endpoints principaux :	
→ POST /api/auth/login	Authentification utilisateur
→ GET /api/pilot/my-flight	Récupérer vol du pilote
→ POST /api/pilot/flights	Créer un nouveau vol
→ GET /api/radar/dashboard	Dashboard contrôleur
→ POST /api/vhf/send	Envoyer message VHF

TABLE 4.2 – API REST endpoints

WebSocket Configuration	
Endpoint	ws://localhost:8080/ws
Protocole	STOMP over WebSocket
Topics :	
· /topic/aircraft/{id}/telemetry	Télémétrie avion spécifique
· /topic/radar/{id}/aircraft-updates	Mises à jour radar
· /user/{username}/queue/flight-updates	Messages privés pilote
· /topic/vhf/{aircraftId}	Communications VHF

TABLE 4.3 – Configuration WebSocket

4.3 Architecture Backend

4.3.1 Structure des Packages

Package	Fichier	Rôle
2*Config	SecurityConfig.java	Configuration sécurité JWT
	KafkaConfig.java	Configuration Kafka
	WebSocketConfig.java	Configuration WebSocket
4*Controller	AuthController.java	Authentification
	PilotDashboardController.java	Dashboard pilote
	RadarDashboardController.java	Dashboard radar
	VhfCommunicationController.java	Communications VHF
5*Service	FlightManagementService.java	Gestion vols
	VhfCommunicationService.java	Communications
	ConflictDetectionService.java	Détection conflits
	RealtimeUpdateService.java	Mises à jour temps réel
	OpenSkyService.java	Intégration OpenSky
5*Model	Aircraft.java	Entité Avion
	Flight.java	Entité Vol
	Pilot.java	Entité Pilote
	VhfMessage.java	Message VHF
	Conflict.java	Conflit détecté
3*Repository	AircraftRepository.java	Repository Avion
	FlightRepository.java	Repository Vol
	PilotRepository.java	Repository Pilote

TABLE 4.4 – Structure des packages backend

4.3.2 Services Principaux

Service	Fonctionnalités
FlightManagementService	→ Création de vols avec validation métier → Gestion des statuts (EN_COURS, TERMINE) → Publication d'événements Kafka
VhfCommunicationService	→ Upload et stockage de fichiers audio → Transcription audio ⇒ texte → Détection d'alertes dans la transcription → Publication Kafka + notification WebSocket
ConflictDetectionService	→ Calcul CPA (Closest Point of Approach) → Détection de conflits entre avions → Génération d'alertes automatiques
RealtimeUpdateService	→ Broadcast positions avions (toutes les 2s) → Consommation Kafka ⇒ WebSocket → Agrégation par batch (500ms)

TABLE 4.5 – Services backend principaux

4.4 Architecture de la Base de Données

4.4.1 Schéma Relationnel

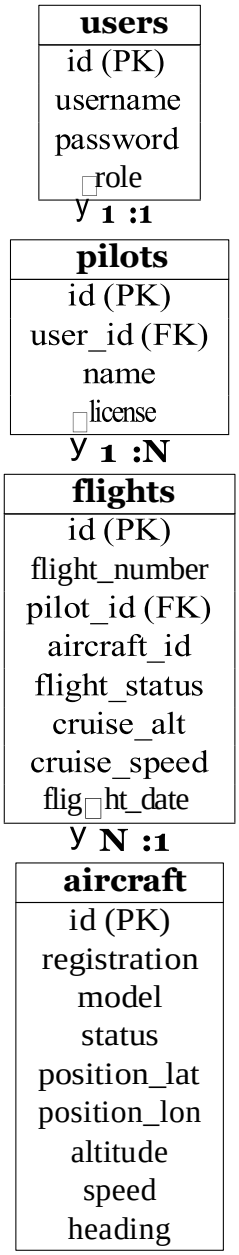


FIGURE 4.2 – Relations entre tables principales

4.4.2 Triggers SQL

Trigger de génération de flight_number

```
CREATE OR REPLACE FUNCTION generate_flight_number()
RETURNS TRIGGER AS $$
BEGIN
    NEW.flight_number := 'AT' || TO_CHAR(NEW.flight_date,
       'YYYYMMDD') ||
                         LPAD(NEXTVAL('flight_sequence')::
                            TEXT, 4, '0');
    NEW.flight_status := 'EN_COURS';
    UPDATE aircraft SET status = 'EN_VOL' WHERE id = NEW.
       aircraft_id;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER flight_creation_trigger
BEFORE INSERT ON flights
FOR EACH ROW
EXECUTE FUNCTION generate_flight_number();
```

TABLE 4.6 – Trigger SQL pour génération automatique de numéros de vol

4.5 Architecture Kafka

4.5.1 Topics Kafka

Topic	Contenu	Producer
topic-flight-events	• flight.created • flight.completed • flight.status.changed	FlightManagementService
topic-aircraft-telemetry	• Positions GPS • Altitude/Vitesse • Cap avion	OpenSkyService ManualTelemetryService
topic-vhf-messages	• Messages audio • Transcriptions • Alertes VHF	VhfCommunicationService
aircraft-alerts	• Alertes conflits • Alertes météo • Notifications	ConflictDetectionService

TABLE 4.7 – Topics Apache Kafka et leurs producteurs

4.6 Architecture WebSocket

4.6.1 Configuration STOMP

Paramètre	Valeur
Endpoint	/ws (avec SockJS fallback)
Protocole	STOMP over WebSocket
Authentification	JWT dans headers CONNECT
Message Broker	Simple broker (/topic, /queue)

TABLE 4.8 – Configuration WebSocket STOMP

4.6.2 Topics WebSocket

Topic/Channel	Description
/topic/aircraft	Diffusion en broadcast des positions de tous les avions
/topic/aircraft/{id}/telemetr	yTélémétrie spécifique à un avion
/topic/radar/{radarCenterId}/	aMirsaafjeoupdaotuers un centre radar spécifique
/user/{username}/queue/flight	-Qpuedaeteps rivée pour les mises à jour de vol d'un pilote
/topic/vhf/{aircraftId}	Communications VHF pour un avion spécifique
/topic/radar/alerts	Diffusion d'alertes radar à tous les contrôleurs

TABLE 4.9 – Topics WebSocket STOMP

4.6.3 Flux de Communication

1. **Connexion** : Client → CONNECT avec JWT
2. **Souscription** : Client → SUBSCRIBE à topics
3. **Reception** : Server → MESSAGE sur topics
4. **Envoi** : Client → SEND pour messages
5. **Déconnexion** : Client → DISCONNECT

4.7 Technologies et Versions

4.7.1 Frontend

Technologie	Version
React	18.2.0
Vite	5.0.8
React Router DOM	6.20.0
Leaflet	1.9.4
React-Leaflet	4.2.1
SockJS	1.6.1
@stomp/stompjs	7.2.1
Axios	1.6.2
Tailwind CSS	3.3.6

TABLE 4.10 – Versions des technologies frontend

4.7.2 Backend

Technologie	Version
Spring Boot	3.2.0
Java	17
PostgreSQL	15
Apache Kafka	3.5.0
Spring WebSocket	(inclus)
Spring Security	(inclus)
JWT (jjwt)	0.11.5

TABLE 4.11 – Versions des technologies backend

Chapitre 5

Implémentation

5.1 Captures d'écran

5.1.1 Page de connexion

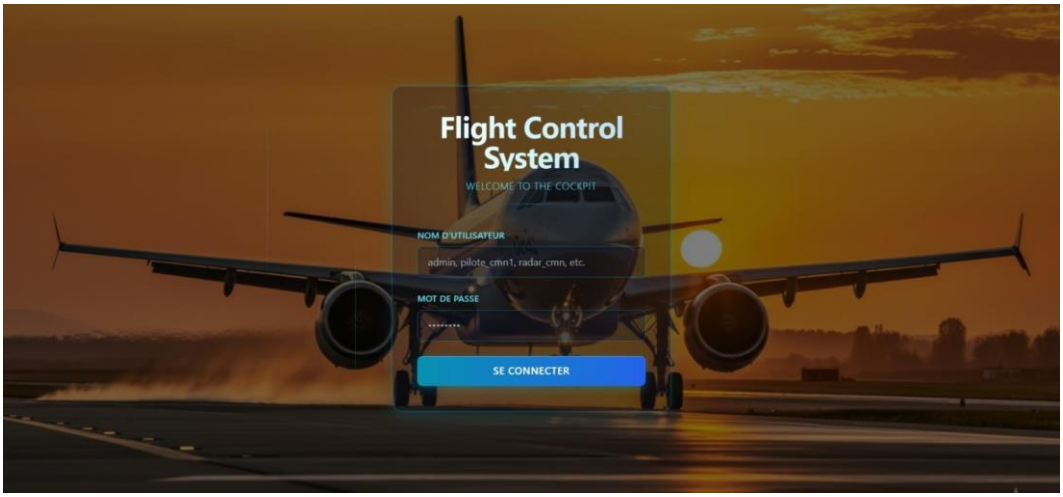


FIGURE 5.1 – Interface de connexion au système

Code backend correspondant :

```java
// Backend: AuthController.java
@PostMapping("/api/auth/login")
public JwtResponse login(@RequestBody AuthRequest request) {
    Authentication auth = authManager.authenticate(...);
    String jwt = jwtProvider.generateToken(auth);
    return new JwtResponse(jwt, userService.getUserDto(auth));
}
```

5.1.2 Cockpit Administrateur - Gestion des vols

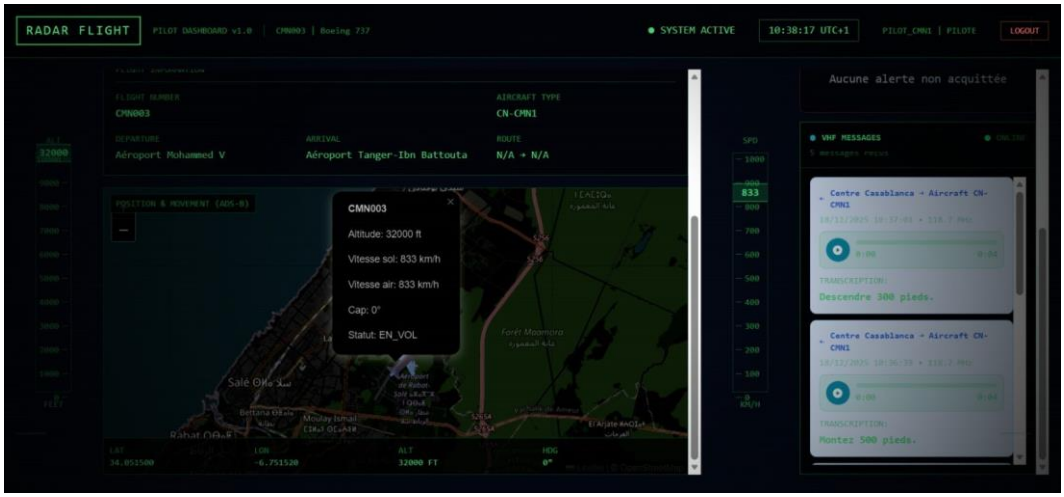


[CAPTURE D'ÉCRAN : Cockpit administrateur]

Code backend correspondant :

```java
// Backend: AdminDashboardController.java
@GetMapping("/api/admin/flights")
public List<FlightDto> getAllFlights() {
    return flightRepository.findAll().stream()
        .map(flightMapper::toDto)
        .collect(Collectors.toList());
}
```

5.1.3 Dashboard Pilote - Radar de vol

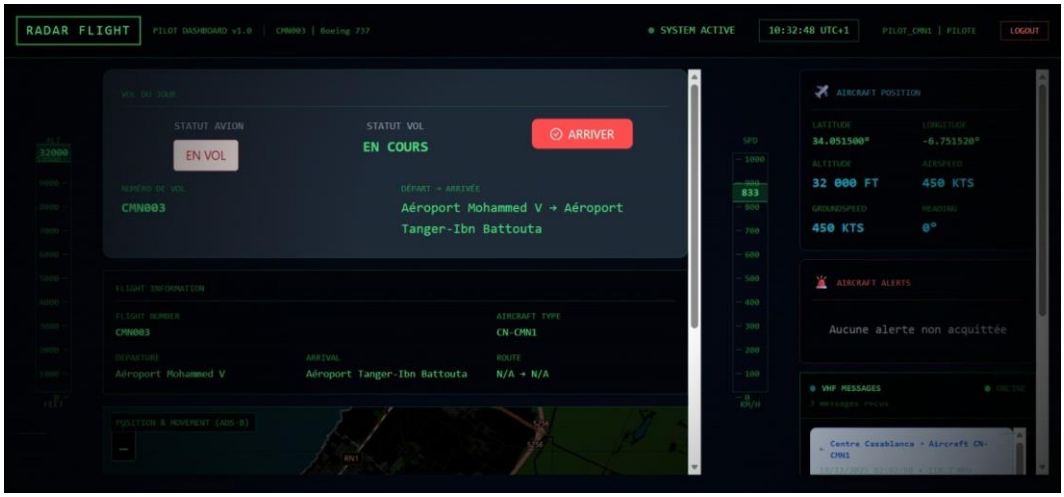


[CAPTURE D'ÉCRAN : Dashboard pilote - Radar]

Code backend correspondant :

```
// Backend: FlightManagementService.java
public void updateAircraftTelemetry(Long aircraftId,
    AircraftTelemetryDto telemetry) {
    Aircraft aircraft = aircraftRepository.findById(aircraftId)
        .orElseThrow(...);
    // ... mise jour coordonnees & statut
    aircraftRepository.save(aircraft);
    realtimeUpdateService.broadcastAircraftPosition(aircraft);
}
```

5.1.4 Dashboard Pilote - Statut du vol

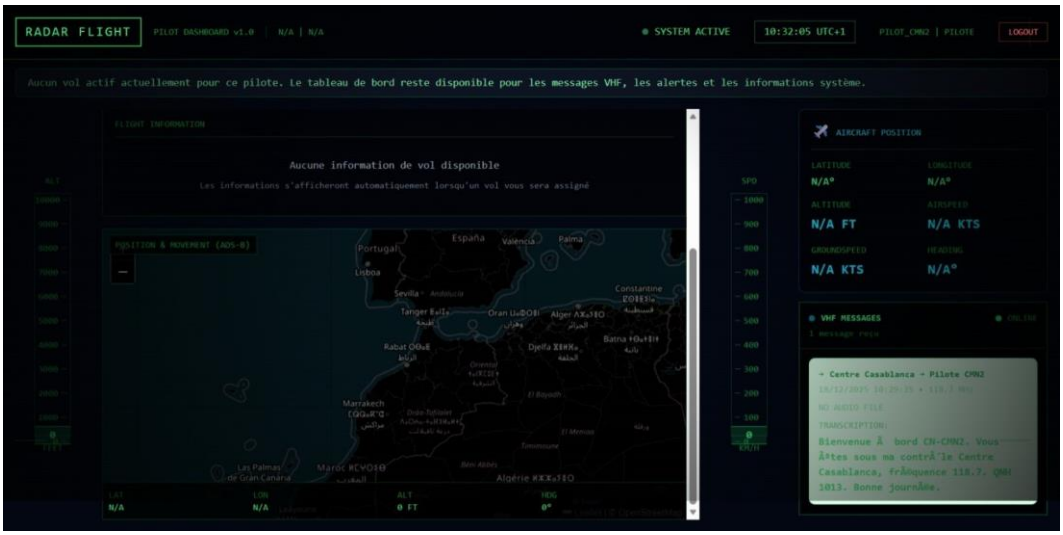


[CAPTURE D'ÉCRAN : Statut du vol]

Code backend correspondant :

```java
// Backend: PilotDashboardController.java
@PostMapping("/api/pilot/flights/{id}/arrive")
public ResponseEntity<?> arriveFlight(@PathVariable Long id) {
    flightService.arriveFlight(id);
    return ResponseEntity.ok().build();
}
```

5.1.5 Dashboard Pilote - Aucun vol actif

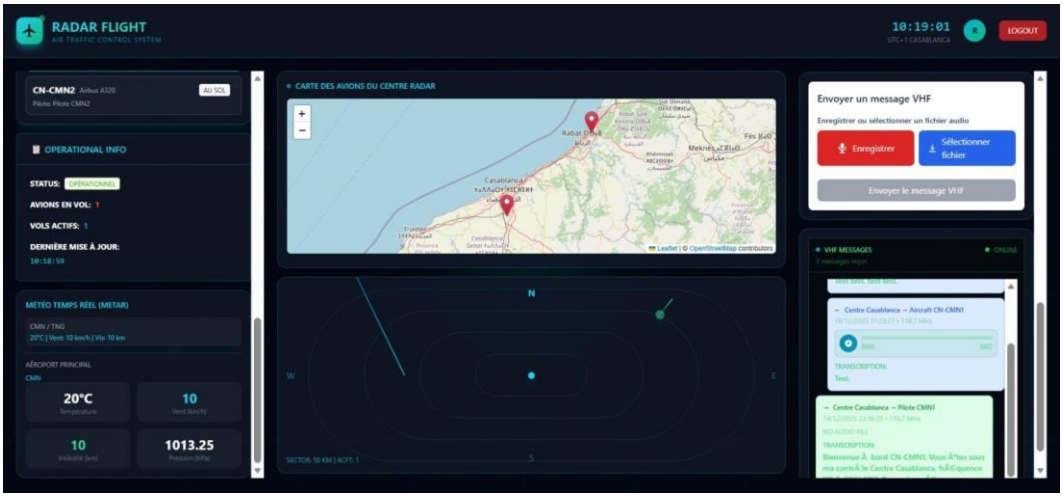


[CAPTURE D'ÉCRAN : Aucun vol actif]

Code frontend correspondant :

```javascript
// Frontend: PilotDashboard.jsx
useEffect(() => {
  api.get('/api/pilot/my-flight')
    .then(res => setFlight(res.data))
    .catch(() => setFlight(null));
}, []);
```

5.1.6 Radar - Carte des avions

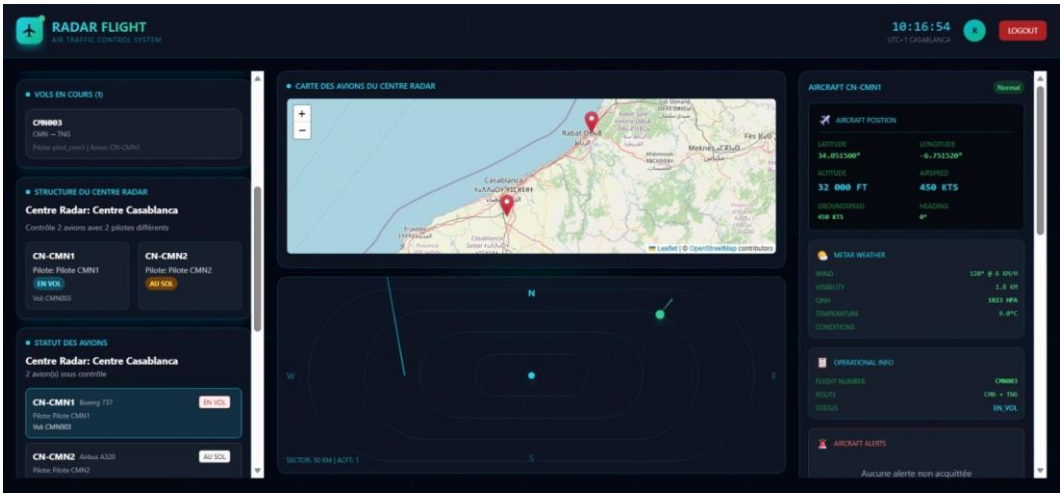


[CAPTURE D'ÉCRAN : Carte radar]

Code backend correspondant :

```java
// Backend: RadarDashboardController.java
@GetMapping("/api/radar/aircraft")
public List<AircraftDto> getAllRadarAircraft() {
    return aircraftRepository.findByRadarCenterId(authRadarId).
        stream()
        .map(aircraftMapper::toDto)
        .collect(Collectors.toList());
}
```

5.1.7 Radar - Positions et alertes

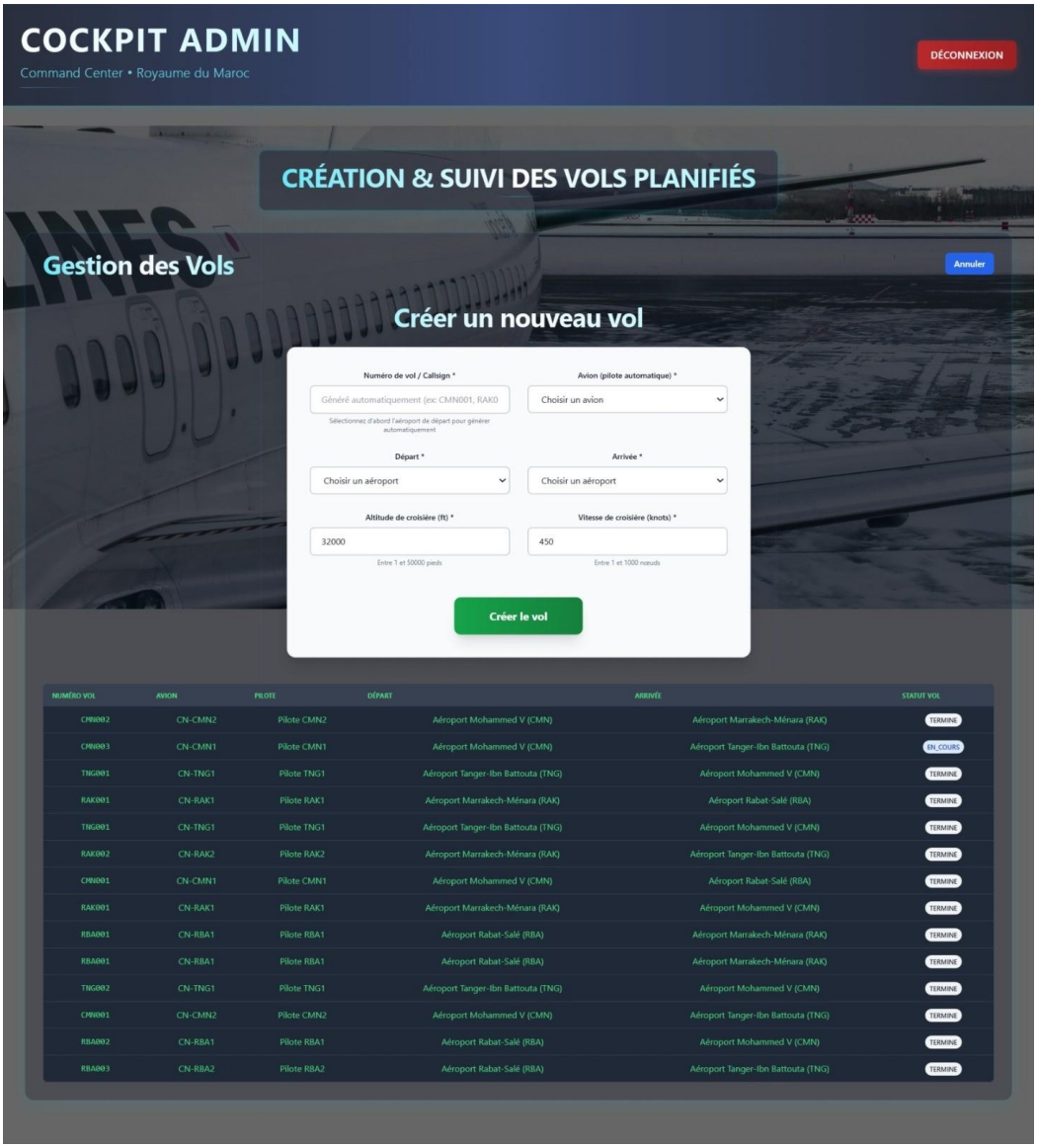


[CAPTURE D'ÉCRAN : Positions et alertes]

Code backend correspondant :

```java
// Backend: RealtimeUpdateService.java
@Scheduled(fixedRate = 2000)
public void broadcastAircraftPositions() {
    List<Aircraft> aircrafts =
        aircraftRepository.findByStatus(AircraftStatus.EN_VOL);
    messagingTemplate.convertAndSend("/topic/aircraft",
        aircrafts);
}
```

5.1.8 Cockpit Admin - Création de vol



[CAPTURE D'ÉCRAN : Formulaire création de vol]

Code backend correspondant :

```java
// Backend: FlightManagementService.java
public Flight createSimplifiedFlight(FlightCreationDto dto) {
    // ... validation
    Flight flight = new Flight();
    // ... set util infos (a roport, pilote, avion)
    return flightRepository.save(flight);
}
```

5.2 Code Source Important

5.2.1 Service de Gestion de Vol

```java
@    Service
@Transactional
public class FlightManagementService {

    @Autowired
    private FlightRepository flightRepository;

    @Autowired
    private AircraftRepository aircraftRepository;

    @Autowired
    private KafkaFlightEventService kafkaFlightEventService;

   public Flight createSimplifiedFlight(Long aircraftId,
                                        Long departureAirportId
                                          ,
                                        Long arrivalAirportId,
                                        Integer cruiseAltitude,
                                       Integer cruiseSpeed,
                                       Long pilotId) {
        // Validation : un pilote ne peut avoir qu'un avion
           EN_VOL
        validatePilotAircraftAssignment(pilotId, aircraftId);

        // Creation entite Flight
        Flight flight = new Flight();
        flight.setAircraftId(aircraftId);
        flight.setDepartureAirportId(departureAirportId);
        flight.setArrivalAirportId(arrivalAirportId);
        flight.setCruiseAltitude(cruiseAltitude);
        flight.setCruiseSpeed(cruiseSpeed);
        flight.setPilotId(pilotId);
        flight.setFlightDate(LocalDate.now());

        // Sauvegarde (trigger SQL genere flight_number)
        Flight savedFlight = flightRepository.save(flight);

```

```
36          // Publication evenement Kafka
37          kafkaFlightEventService.publishFlightCreatedEvent(
                savedFlight);
38
39          return savedFlight;
40      }
41
42      private void validatePilotAircraftAssignment(Long pilotId,
            Long aircraftId) {
43          // Verifier pilote n'a pas deja un avion EN_VOL
44          List<Flight> activeFlights = flightRepository
45              .findByPilotIdAndFlightStatus(pilotId, FlightStatus.
                    EN_COURS);
46          if (!activeFlights.isEmpty()) {
47              throw new BusinessException("Le pilote a deja un vol
                    en cours");
48          }
49
50          // Verifier avion est AU_SOL
51          Aircraft aircraft = aircraftRepository.findById(
                aircraftId)
52              .orElseThrow(() -> new NotFoundException("Avion non
                    trouve"));
53          if (aircraft.getStatus() != AircraftStatus.AU_SOL) {
54              throw new BusinessException("L'avion n'est pas au
                    sol");
55          }
56      }
57  }
```

5.2.2 Service de Détection de Conflits

```
1   @Service
2   public class ConflictDetectionService {
3
4       private static final double HORIZONTAL_THRESHOLD_M = 5000.0;
            // 5 km
5       private static final double VERTICAL_THRESHOLD_M = 300.0; //
            300 m
6
7       @Autowired
8       private AircraftRepository aircraftRepository;
9
10      @Autowired
11      private ConflictRepository conflictRepository;
12
13      @Scheduled(fixedRate = 5000) // Toutes les 5 secondes
14      @Transactional
15      public void detectConflicts() {
16          List<Aircraft> aircraftInFlight = aircraftRepository
```

```
            .findByStatus(AircraftStatus.EN_VOL);

        for (int i = 0; i < aircraftInFlight.size(); i++) {
            for (int j = i + 1; j < aircraftInFlight.size(); j
                ++) {
                detectConflictBetweenAircraft(
                    aircraftInFlight.get(i),
                    aircraftInFlight.get(j)
                );
            }
        }
    }

    private void detectConflictBetweenAircraft(Aircraft a1,
        Aircraft a2) {
        CPAResult cpa = calculateCPA(a1, a2);

        if (cpa.getHorizontalDistance() < HORIZONTAL_THRESHOLD_M
            &&
            cpa.getVerticalDistance() < VERTICAL_THRESHOLD_M) {
            // Creer conflit
            Conflict conflict = new Conflict();
            conflict.setAircraftAId(a1.getId());
            conflict.setAircraftBId(a2.getId());
            conflict.setHorizontalDistance(cpa.
                getHorizontalDistance());
            conflict.setVerticalDistance(cpa.getVerticalDistance
                ());
            conflict.setStatus(ConflictStatus.SENT);
            conflictRepository.save(conflict);

            // Publier alerte Kafka
            kafkaAlertService.createAlert(a1.getId(),
                "Conflit detecte avec " + a2.getRegistration());
        }
    }

    private CPAResult calculateCPA(Aircraft a1, Aircraft a2) {
        // Calcul du Closest Point of Approach
        // Formules geometriques pour predire la distance
            minimale future
        // entre deux avions en mouvement
        // ...
        return cpaResult;
    }
}
```

5.2.3 Hook WebSocket Frontend

```
// hooks/useWebSocket.js
```

```
import { useEffect, useState, useRef } from 'react';
import SockJS from 'sockjs-client';
import { Client } from '@stomp/stompjs';

export function useWebSocket(topic, onMessage) {
  const [connected, setConnected] = useState(false);
  const clientRef = useRef(null);

  useEffect(() => {
    const socket = new SockJS('http://localhost:8080/ws');
    const client = new Client({
      webSocketFactory: () => socket,
      reconnectDelay: 5000,
      heartbeatIncoming: 4000,
      heartbeatOutgoing: 4000,
      connectHeaders: {
        Authorization: `Bearer ${localStorage.getItem('token')}`
      },
      onConnect: () => {
        setConnected(true);
        client.subscribe(topic, (message) => {
          try {
            const data = JSON.parse(message.body);
            onMessage(data);
          } catch (error) {
            console.error('Erreur parsing message WebSocket:',
                error);
          }
        });
      },
      onDisconnect: () => {
        setConnected(false);
      }
    });

    client.activate();
    clientRef.current = client;

    return () => {
      if (clientRef.current) {
        clientRef.current.deactivate();
      }
    };
  }, [topic, onMessage]);

  return { connected };
}
```

5.2.4 Tableau récapitulatif des captures

Interface	Rôle	Fonctionnalités principales
Page de connexion	Authentification	Génération JWT, gestion rôles
Cockpit Admin	Administration	Liste vols, gestion utilisateurs
Dashboard Pilote	Opérations pilote	Visualisation vol, communications
Radar Contrôleur	Surveillance	Carte avions, détection conflits
Création de vol	Planification	Formulaire création, validation

TABLE 5.1 – Récapitulatif des interfaces implémentées

Conclusion Générale

Le projet FlightRadarPFA a permis de concevoir et d'implémenter un système robuste et modulaire de suivi de vols en temps réel, parfaitement adapté aux besoins des utilisateurs identifiés. L'architecture se distingue par une séparation nette entre les couches frontend et backend, favorisant la maintenabilité et l'évolutivité de la solution. La communication entre le client et le serveur repose principalement sur les API REST pour les opérations synchrones et sur WebSocket pour l'acheminement instantané des données critiques, telles que la position des avions ou les messages VHF. Ce choix garantit une expérience utilisateur fluide, essentielle dans un contexte métier où la réactivité est primordiale.

Un autre pilier de fiabilité du système réside dans l'intégration d'Apache Kafka : cette plateforme de messagerie assure la résilience face aux pics de charge ainsi que la persistance des événements majeurs (télémétrie, alertes, communications). La sécurité est également un atout du projet : l'utilisation généralisée de JWT pour l'authentification et l'application stricte d'autorisations par rôles (pilote, contrôleur, admin) empêchent les accès non contrôlés aux fonctionnalités sensibles.

Malgré ces forces, le développement a rencontré plusieurs limites notables. La dépendance à l'API OpenSky, soumise à une fréquence d'interrogation restreinte, limite la fraîcheur des données de vol ; la gestion des flux WebSocket, bien que performante grâce à l'agrégation par batch, requiert des optimisations constantes afin de rester efficiente lors de la montée en charge. De plus, la détection automatique des conflits, fondée sur le calcul du CPA (Closest Point of Approach), s'avère mathématiquement complexe et devra être soigneusement optimisée pour garantir la réactivité en cas d'accroissement du trafic simulé.

Pour répondre à ces défis, de nombreuses perspectives d'amélioration sont envisagées. À court terme, l'introduction d'un cache Redis visera à accélérer la délivrance des informations fréquemment consultées (positions des avions, météo). Par ailleurs, le stockage de l'historique de télémétrie dans une base de données time-series (InfluxDB) offrira de nouvelles possibilités d'analyse et de visualisation. L'implémentation d'un pipeline ADS-B direct via RTL-SDR permettrait de s'affranchir des limitations des API externes, en rendant la plateforme totalement autonome pour la collecte des données de positionnement aérien. Enfin, la transformation progressive de l'architecture vers un modèle microservices, associée à la mise en place de solutions de monitoring modernes (Prometheus, Grafana) et d'un pipeline CI/CD, constitueront des leviers majeurs pour accroître la scalabilité, la maintenabilité et l'observabilité du système à long terme.

Bibliographie

[1] Pivotal Software. *Spring Boot Reference Documentation*. Spring Framework, 2023. Disponible sur : https://docs.spring.io/spring-boot/docs/current/reference/html/

[2] Meta Platforms. *React Documentation*. React.dev, 2023. Disponible sur : https://react.dev/

[3] PostgreSQL Global Development Group. *PostgreSQL 15 Documentation*. PostgreSQL.org, 2022. Disponible sur : https://www.postgresql.org/docs/15/

[4] Apache Software Foundation. *Apache Kafka Documentation*. Kafka.apache.org, 2023. Disponible sur : https://kafka.apache.org/documentation/

[5] Spring Framework Team. *Spring WebSocket Documentation*. Spring.io, 2023. Disponible sur : https://docs.spring.io/spring-framework/reference/web/websocket.html

[6] OpenSky Network. *OpenSky Network API Documentation*. OpenSky-network.org, 2023. Disponible sur : https://openskynetwork.github.io/opensky-api/

[7] Leaflet Contributors. *Leaflet - an open-source JavaScript library for mobile-friendly interactive maps*. Leafletjs.com, 2023. Disponible sur : https://leafletjs.com/

[8] Oracle Corporation. *Java Platform, Standard Edition Documentation - Java 17*. Oracle.com, 2021. Disponible sur : https://docs.oracle.com/en/java/javase/17/

[9] IETF. *RFC 7519 : JSON Web Token (JWT)*. Internet Engineering Task Force, Mai 2015. Disponible sur : https://tools.ietf.org/html/rfc7519

[10] OWASP Foundation. *OWASP Top 10 - 2021 : The Ten Most Critical Web Application Security Risks*. OWASP.org, 2021. Disponible sur : https://owasp.org/www-project-top-ten/

[11] European Organisation for the Safety of Air Navigation (EUROCONTROL). *ADS-B Implementation and Operations Guidance Document*. Edition 3.0, 2017. Disponible sur : https://www.eurocontrol.int/

[12] Federal Aviation Administration (FAA). *Automatic Dependent Surveillance-Broadcast (ADS-B) Out Performance Requirements*. Federal Register, Vol. 75, No. 103, 2010.

[13] Fowler, M. *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional, 2002. ISBN : 978-0321127426

[14] Newman, S. *Building Microservices : Designing Fine-Grained Systems*. O'Reilly Media, 2nd Edition, 2021. ISBN : 978-1492034025

[15] Hohpe, G., Woolf, B. *Enterprise Integration Patterns : Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley Professional, 2003. ISBN : 978-0321200686

[16] Crockford, D. *JavaScript : The Good Parts*. O'Reilly Media, 2008. ISBN : 978-0596517748

[17] Walls, C. *Spring in Action*. Manning Publications, 6th Edition, 2021. ISBN : 978-1617297571

[18] PostgreSQL Global Development Group. *PostgreSQL : Advanced Server Programming*. PostgreSQL.org, 2022. Disponible sur : https://www.postgresql.org/docs/15/plpgsql.html

[19] Apache Software Foundation. *Kafka : A Distributed Streaming Platform*. Proceedings of the VLDB Endowment, Vol. 8, No. 12, 2015.

[20] W3C. *WebSocket API*. W3C Recommendation, 2012. Disponible sur : https://www.w3.org/TR/websockets/

[21] International Civil Aviation Organization (ICAO). *Annex 10 to the Convention on International Civil Aviation - Aeronautical Telecommunications*. ICAO, Volume IV, 7th Edition, 2018.

[22] React Team. *React 18 : The Plan for React 18*. React.dev Blog, 2021. Disponible sur : https://react.dev/blog/2021/06/15/the-plan-for-react-18

[23] Spring Framework Team. *Spring Security Reference Documentation*. Spring.io, 2023. Disponible sur : https://docs.spring.io/spring-security/reference/

[24] Vite Team. *Vite - Next Generation Frontend Tooling*. Vitejs.dev, 2023. Disponible sur : https://vitejs.dev/

[25] Docker Inc. *Docker Documentation*. Docker.com, 2023. Disponible sur : https://docs.docker.com/

[26] Kubernetes Authors. *Kubernetes Documentation*. Kubernetes.io, 2023. Disponible sur : https://kubernetes.io/docs/

[27] Redis Labs. *Redis Documentation*. Redis.io, 2023. Disponible sur : https://redis.io/docs/

[28] InfluxData. *InfluxDB Documentation*. Influxdata.com, 2023. Disponible sur : https://docs.influxdata.com/influxdb/

[29] RTL-SDR Community. *RTL-SDR Blog : Software Defined Radio*. Rtl-sdr.com, 2023. Disponible sur : https://www.rtl-sdr.com/

[30] International Air Transport Association (IATA). *IATA Airport Codes*. IATA.org, 2023. Disponible sur : https://www.iata.org/

.1 Extraits de code source complets

1.1 Configuration Spring Security (SecurityConfig.java)

Listing 1 – Configuration Spring Security

```
1 package com.flightradar.config;
2
```

```
3  import com.flightradar.security.JwtAuthenticationFilter;
4  import com.flightradar.security.JwtAuthenticationEntryPoint;
5  import org.springframework.beans.factory.annotation.Autowired;
6  import org.springframework.context.annotation.Bean;
7  import org.springframework.context.annotation.Configuration;
8  import org.springframework.security.authentication.
       AuthenticationManager;
9  import org.springframework.security.config.annotation.
       authentication.configuration.AuthenticationConfiguration;
10 import org.springframework.security.config.annotation.method.
       configuration.EnableMethodSecurity;
11 import org.springframework.security.config.annotation.web.
       builders.HttpSecurity;
12 import org.springframework.security.config.annotation.web.
       configuration.EnableWebSecurity;
13 import org.springframework.security.config.http.
       SessionCreationPolicy;
14 import org.springframework.security.crypto.bcrypt.
       BCryptPasswordEncoder;
15 import org.springframework.security.crypto.password.
       PasswordEncoder;
16 import org.springframework.security.web.SecurityFilterChain;
17 import org.springframework.security.web.authentication.
       UsernamePasswordAuthenticationFilter;
18
19 @Configuration
20 @EnableWebSecurity
21 @EnableMethodSecurity
22 public class SecurityConfig {
23
24     @Autowired
25     private JwtAuthenticationEntryPoint
           jwtAuthenticationEntryPoint;
26
27     @Autowired
28     private JwtAuthenticationFilter jwtAuthenticationFilter;
29
30     @Bean
31     public PasswordEncoder passwordEncoder() {
32         return new BCryptPasswordEncoder();
33     }
34
35     @Bean
36     public AuthenticationManager authenticationManager(
37             AuthenticationConfiguration authConfig) throws
                Exception {
38         return authConfig.getAuthenticationManager();
39     }
40
41     @Bean
```

```
42      public SecurityFilterChain filterChain(HttpSecurity http)
            throws Exception {
43          http.csrf(csrf -> csrf.disable())
44              .authorizeHttpRequests(auth -> auth
45                  .requestMatchers("/api/auth/**").permitAll()
46                  .requestMatchers("/api/airports/**").permitAll()
47                  .requestMatchers("/api/weather/**").permitAll()
48                  .requestMatchers("/ws/**").permitAll()
49                  .requestMatchers("/api/pilot/**").hasAnyRole("
                        PILOTE", "ADMIN")
50                  .requestMatchers("/api/radar/**").hasAnyRole("
                        CENTRE_RADAR", "ADMIN")
51                  .anyRequest().authenticated()
52              )
53              .exceptionHandling(ex -> ex
54                  .authenticationEntryPoint(
                        jwtAuthenticationEntryPoint)
55              )
56              .sessionManagement(session -> session
57                  .sessionCreationPolicy(SessionCreationPolicy.
                        STATELESS)
58              );
59
60          http.addFilterBefore(jwtAuthenticationFilter,
61                               UsernamePasswordAuthenticationFilter.
                                    class);
62
63          return http.build();
64      }
65  }
```

1.2 Configuration Kafka (KafkaConfig.java)

Listing 2 – Configuration Kafka

```
1   package com.flightradar.config;
2
3   import org.apache.kafka.clients.consumer.ConsumerConfig;
4   import org.apache.kafka.clients.producer.ProducerConfig;
5   import org.apache.kafka.common.serialization.StringDeserializer;
6   import org.apache.kafka.common.serialization.StringSerializer;
7   import org.springframework.beans.factory.annotation.Value;
8   import org.springframework.context.annotation.Bean;
9   import org.springframework.context.annotation.Configuration;
10  import org.springframework.kafka.config.
        ConcurrentKafkaListenerContainerFactory;
11  import org.springframework.kafka.core.*;
12  import org.springframework.kafka.support.serializer.
        JsonDeserializer;
13  import org.springframework.kafka.support.serializer.
        JsonSerializer;
```

```
14
15 import java.util.HashMap;
16 import java.util.Map;
17
18 @Configuration
19 public class KafkaConfig {
20
21     @Value("${spring.kafka.bootstrap-servers}")
22     private String bootstrapServers;
23
24     @Bean
25     public ProducerFactory<String, Object> producerFactory() {
26         Map<String, Object> configProps = new HashMap<>();
27         configProps.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG,
               bootstrapServers);
28         configProps.put(ProducerConfig.
               KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class);
29         configProps.put(ProducerConfig.
               VALUE_SERIALIZER_CLASS_CONFIG, JsonSerializer.class);
30         configProps.put(ProducerConfig.ACKS_CONFIG, "all");
31         configProps.put(ProducerConfig.COMPRESSION_TYPE_CONFIG,
               "gzip");
32         configProps.put(ProducerConfig.BATCH_SIZE_CONFIG, 16384)
               ;
33         configProps.put(ProducerConfig.LINGER_MS_CONFIG, 20);
34         configProps.put(ProducerConfig.RETRIES_CONFIG, 3);
35
36         return new DefaultKafkaProducerFactory<>(configProps);
37     }
38
39     @Bean
40     public KafkaTemplate<String, Object> kafkaTemplate() {
41         return new KafkaTemplate<>(producerFactory());
42     }
43
44     @Bean
45     public ConsumerFactory<String, Object> consumerFactory() {
46         Map<String, Object> props = new HashMap<>();
47         props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG,
               bootstrapServers);
48         props.put(ConsumerConfig.GROUP_ID_CONFIG, "flightradar-
               group");
49         props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG,
               StringDeserializer.class);
50         props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG
               , JsonDeserializer.class);
51         props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "
               earliest");
52         props.put(JsonDeserializer.TRUSTED_PACKAGES, "*");
53
54         return new DefaultKafkaConsumerFactory<>(props);
```

```
55      }
56
57      @Bean
58      public ConcurrentKafkaListenerContainerFactory<String,
           Object> kafkaListenerContainerFactory() {
59          ConcurrentKafkaListenerContainerFactory<String, Object>
               factory =
60                  new ConcurrentKafkaListenerContainerFactory<>();
61          factory.setConsumerFactory(consumerFactory());
62          return factory;
63      }
64  }
```

1.3 Hook WebSocket React (useWebSocket.js)

Listing 3 – Hook WebSocket React

```
1  import { useEffect, useRef, useState, useCallback } from 'react
       ';
2  import SockJS from 'sockjs-client';
3  import { Client } from '@stomp/stompjs';
4
5  export const useWebSocket = (url, topics, onMessage) => {
6    const [connected, setConnected] = useState(false);
7    const [error, setError] = useState(null);
8    const clientRef = useRef(null);
9    const reconnectTimeoutRef = useRef(null);
10
11   const connect = useCallback(() => {
12     if (clientRef.current?.connected) {
13       return;
14     }
15
16     const socket = new SockJS(url);
17     const client = new Client({
18       webSocketFactory: () => socket,
19       reconnectDelay: 5000,
20       heartbeatIncoming: 4000,
21       heartbeatOutgoing: 4000,
22
23       onConnect: () => {
24         console.log('WebSocket connected');
25         setConnected(true);
26         setError(null);
27
28         // Subscribe to topics
29         topics.forEach(topic => {
30           client.subscribe(topic, (message) => {
31             try {
32               const data = JSON.parse(message.body);
33               onMessage(topic, data);
```

```
            } catch (err) {
              console.error('Error parsing WebSocket message:',
                 err);
            }
          });
        });
      },

      onDisconnect: () => {
        console.log('WebSocket disconnected');
        setConnected(false);
      },

      onStompError: (frame) => {
        console.error('STOMP error:', frame);
        setError(frame.headers['message'] || 'WebSocket error');
      },
    });

    client.activate();
    clientRef.current = client;
  }, [url, topics, onMessage]);

  const disconnect = useCallback(() => {
    if (reconnectTimeoutRef.current) {
      clearTimeout(reconnectTimeoutRef.current);
    }

    if (clientRef.current) {
      clientRef.current.deactivate();
      clientRef.current = null;
    }

    setConnected(false);
  }, []);

  useEffect(() => {
    connect();

    return () => {
      disconnect();
    };
  }, [connect, disconnect]);

  return { connected, error, disconnect, reconnect: connect };
};
```

.2 Scripts SQL

2.1 Schéma de base de données complet

Listing 4 – Schéma de base de données complet

```sql
-- Table users
CREATE TABLE users (
    id BIGSERIAL PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    role VARCHAR(20) NOT NULL CHECK (role IN ('ADMIN', 'PILOTE',
        'CENTRE_RADAR')),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Table pilots
CREATE TABLE pilots (
    id BIGSERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    license VARCHAR(50) UNIQUE NOT NULL,
    experience_years INTEGER NOT NULL DEFAULT 0,
    user_id BIGINT,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET
       NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Table airports
CREATE TABLE airports (
    id BIGSERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    city VARCHAR(100) NOT NULL,
    code_iata VARCHAR(3) UNIQUE NOT NULL,
    latitude DECIMAL(10, 8) NOT NULL,
    longitude DECIMAL(11, 8) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Table runways
CREATE TABLE runways (
    id BIGSERIAL PRIMARY KEY,
    name VARCHAR(10) NOT NULL,
    orientation DECIMAL(5, 2) NOT NULL CHECK (orientation >= 0
       AND orientation < 360),
    length_meters INTEGER NOT NULL,
    width_meters INTEGER NOT NULL,
    airport_id BIGINT NOT NULL,
    FOREIGN KEY (airport_id) REFERENCES airports(id) ON DELETE
       CASCADE,
    UNIQUE(airport_id, name)
```

```
42 );
43
44 -- Table radar_centers
45 CREATE TABLE radar_centers (
46     id BIGSERIAL PRIMARY KEY,
47     name VARCHAR(100) NOT NULL,
48     code VARCHAR(20) UNIQUE NOT NULL,
49     frequency DECIMAL(6, 2) NOT NULL,
50     airport_id BIGINT NOT NULL,
51     user_id BIGINT,
52     FOREIGN KEY (airport_id) REFERENCES airports(id) ON DELETE
           CASCADE,
53     FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET
           NULL,
54     UNIQUE(airport_id)
55 );
56
57 -- Table aircraft
58 CREATE TABLE aircraft (
59     id BIGSERIAL PRIMARY KEY,
60     model VARCHAR(50) NOT NULL,
61     registration VARCHAR(20) UNIQUE NOT NULL,
62     status VARCHAR(20) NOT NULL CHECK (status IN ('AU_SOL', '
           DECOLLAGE', 'EN_VOL', 'ATTERRISSAGE', 'EN_ATTENTE')),
63     airport_id BIGINT,
64     pilot_id BIGINT,
65     radar_center_id BIGINT,
66     position_lat DECIMAL(10, 8),
67     position_lon DECIMAL(11, 8),
68     altitude DECIMAL(10, 2) DEFAULT 0,
69     speed DECIMAL(8, 2) DEFAULT 0,
70     heading DECIMAL(5, 2) DEFAULT 0 CHECK (heading >= 0 AND
           heading < 360),
71     air_speed DECIMAL(8, 2),
72     last_update TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
73     FOREIGN KEY (airport_id) REFERENCES airports(id) ON DELETE
           SET NULL,
74     FOREIGN KEY (pilot_id) REFERENCES pilots(id) ON DELETE SET
           NULL,
75     FOREIGN KEY (radar_center_id) REFERENCES radar_centers(id)
           ON DELETE SET NULL
76 );
77
78 -- Table flights
79 CREATE TABLE flights (
80     id BIGSERIAL PRIMARY KEY,
81     flight_number VARCHAR(10) UNIQUE NOT NULL,
82     callsign VARCHAR(20) UNIQUE,
83     aircraft_id BIGINT NOT NULL,
84     departure_airport_id BIGINT NOT NULL,
85     arrival_airport_id BIGINT NOT NULL,
```

```
86      flight_status VARCHAR(20) NOT NULL CHECK (flight_status IN (
            'EN_COURS', 'TERMINE')),
87      cruise_altitude INTEGER,
88      cruise_speed INTEGER,
89      alternate_airport_id BIGINT,
90      pilot_id BIGINT,
91      flight_date DATE,
92      radar_center_id BIGINT,
93      actual_departure TIMESTAMP,
94      actual_arrival TIMESTAMP,
95      created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
96      FOREIGN KEY (aircraft_id) REFERENCES aircraft(id) ON DELETE
            CASCADE,
97      FOREIGN KEY (departure_airport_id) REFERENCES airports(id)
            ON DELETE RESTRICT,
98      FOREIGN KEY (arrival_airport_id) REFERENCES airports(id) ON
            DELETE RESTRICT
99  );
100
101 -- Table weather_data
102 CREATE TABLE weather_data (
103     id BIGSERIAL PRIMARY KEY,
104     airport_id BIGINT NOT NULL,
105     wind_speed DECIMAL(6, 2) NOT NULL,
106     wind_direction DECIMAL(5, 2) NOT NULL CHECK (wind_direction
            >= 0 AND wind_direction < 360),
107     visibility DECIMAL(6, 2) NOT NULL,
108     temperature DECIMAL(5, 2) NOT NULL,
109     humidity INTEGER NOT NULL CHECK (humidity >= 0 AND humidity
            <= 100),
110     pressure DECIMAL(7, 2) NOT NULL,
111     conditions VARCHAR(50) NOT NULL,
112     crosswind DECIMAL(6, 2) DEFAULT 0,
113     alert BOOLEAN DEFAULT FALSE,
114     timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
115     FOREIGN KEY (airport_id) REFERENCES airports(id) ON DELETE
            CASCADE
116 );
117
118 -- Table communications
119 CREATE TABLE communications (
120     id BIGSERIAL PRIMARY KEY,
121     sender_type VARCHAR(20) NOT NULL CHECK (sender_type IN ('
            RADAR', 'AIRCRAFT', 'AIRPORT')),
122     sender_id BIGINT NOT NULL,
123     receiver_type VARCHAR(20) NOT NULL CHECK (receiver_type IN (
            'RADAR', 'AIRCRAFT', 'AIRPORT')),
124     receiver_id BIGINT NOT NULL,
125     message TEXT NOT NULL,
126     frequency DECIMAL(6, 2) NOT NULL DEFAULT 121.5,
127     timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
);

-- Index pour optimiser les requ tes  fr quentes
CREATE INDEX idx_aircraft_status ON aircraft(status);
CREATE INDEX idx_aircraft_pilot ON aircraft(pilot_id);
CREATE INDEX idx_flights_status ON flights(flight_status);
CREATE INDEX idx_flights_aircraft ON flights(aircraft_id);
CREATE INDEX idx_weather_airport ON weather_data(airport_id);
CREATE INDEX idx_weather_timestamp ON weather_data(timestamp
    DESC);
```

2.2 Séquences PostgreSQL pour génération de numéros de vol

Listing 5 – Séquences PostgreSQL

```
-- Cr ation des s quences pour chaque a roport
CREATE SEQUENCE IF NOT EXISTS flight_seq_CMN START 1;
CREATE SEQUENCE IF NOT EXISTS flight_seq_RBA START 1;
CREATE SEQUENCE IF NOT EXISTS flight_seq_RAK START 1;
CREATE SEQUENCE IF NOT EXISTS flight_seq_TNG START 1;
```

.3 Configuration Docker Compose

3.1 docker-compose.yml (Architecture proposée)

Listing 6 – Configuration Docker Compose

```
version: '3.8'

services:
  postgres:
    image: postgres:15
    container_name: flightradar-postgres
    environment:
      POSTGRES_DB: flightradar
      POSTGRES_USER: flightradar
      POSTGRES_PASSWORD: flightradar123
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - flightradar-network

  kafka:
    image: confluentinc/cp-kafka:latest
    container_name: flightradar-kafka
    environment:
      KAFKA_BROKER_ID: 1
```

```
23        KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
24        KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
25        KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
26      ports:
27        - "9092:9092"
28      depends_on:
29        - zookeeper
30      networks:
31        - flightradar-network
32
33    zookeeper:
34      image: confluentinc/cp-zookeeper:latest
35      container_name: flightradar-zookeeper
36      environment:
37        ZOOKEEPER_CLIENT_PORT: 2181
38        ZOOKEEPER_TICK_TIME: 2000
39      ports:
40        - "2181:2181"
41      networks:
42        - flightradar-network
43
44    redis:
45      image: redis:7-alpine
46      container_name: flightradar-redis
47      ports:
48        - "6379:6379"
49      volumes:
50        - redis_data:/data
51      networks:
52        - flightradar-network
53
54    influxdb:
55      image: influxdb:2.7
56      container_name: flightradar-influxdb
57      environment:
58        DOCKER_INFLUXDB_INIT_MODE: setup
59        DOCKER_INFLUXDB_INIT_USERNAME: admin
60        DOCKER_INFLUXDB_INIT_PASSWORD: admin123
61        DOCKER_INFLUXDB_INIT_ORG: flightradar
62        DOCKER_INFLUXDB_INIT_BUCKET: telemetry
63      ports:
64        - "8086:8086"
65      volumes:
66        - influxdb_data:/var/lib/influxdb2
67      networks:
68        - flightradar-network
69
70    backend:
71      build:
72        context: ./backend
73        dockerfile: Dockerfile
```

```
    container_name: flightradar-backend
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/
          flightradar
      SPRING_DATASOURCE_USERNAME: flightradar
      SPRING_DATASOURCE_PASSWORD: flightradar123
      SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:9092
      SPRING_REDIS_HOST: redis
      SPRING_REDIS_PORT: 6379
    ports:
      - "8080:8080"
    depends_on:
      - postgres
      - kafka
      - redis
    networks:
      - flightradar-network

  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    container_name: flightradar-frontend
    ports:
      - "3000:80"
    depends_on:
      - backend
    networks:
      - flightradar-network

volumes:
  postgres_data:
  redis_data:
  influxdb_data:

networks:
  flightradar-network:
    driver: bridge
```

.4 Configuration Frontend (package.json)

4.1 Dépendances principales

Listing 7 – Dépendances Frontend

```
{
  "name": "flightradar-frontend",
  "version": "1.0.0",
  "private": true,
  "dependencies": {
```

```
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-router-dom": "^6.20.0",
    "leaflet": "^1.9.4",
    "react-leaflet": "^4.2.1",
    "sockjs-client": "^1.6.1",
    "@stomp/stompjs": "^7.2.1",
    "axios": "^1.6.2",
    "antd": "^5.29.1",
    "chart.js": "^4.5.1",
    "react-chartjs-2": "^5.3.1"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.2.1",
    "vite": "^5.0.8",
    "tailwindcss": "^3.3.6",
    "autoprefixer": "^10.4.16",
    "postcss": "^8.4.32"
  },
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  }
}
```

.5 Exemples de messages Kafka

5.1 Événement de création de vol

Listing 8 – Événement de création de vol

```
{
  "eventType": "flight.created",
  "  flightId  ":        123  ,
  "flightNumber":  "CMN001",
  "aircraftId": 45,
  "pilotId": 12,
  "departureAirportId": 1,
  "  arrivalAirportId  ":        2  ,
  " flightStatus ":    " EN_COURS ",
  "timestamp": "2024-01-15T10:30:00Z"
}
```

5.2 Message de télémétrie d'avion

Listing 9 – Télémétrie d'avion

```
{
```

```
  "aircraftId": 45,
  "registration": "CN-AT06",
  "position": {
    "latitude": 33.367500,
    "longitude": -7.589800
  },
  "altitude": 35000,
  "speed": 850,
  "heading": 180,
  "timestamp": "2024-01-15T10:30:05Z"
}
```

5.3 Message VHF

Listing 10 – Message VHF

```
{
  "id": 789,
  "aircraftId": 45,
  "radarCenterId": 1,
  "audioUrl": "/audio/vhf/789.wav",
  "transcription": "CMN Tower, CN-AT06 requesting clearance for
     takeoff",
  "fromName": "CN-AT06",
  "toName": "CMN Tower",
  "frequency": 121.5,
  "timestamp": "2024-01-15T10:30:10Z",
  "alerts": []
}
```

.6 Format des réponses API REST

6.1 Réponse de connexion (POST /api/auth/login)

Listing 11 – Réponse de connexion

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": 12,
    "username": "pilot01",
    "role": "PILOTE",
    "pilot": {
      "id": 5,
      "name": "Ahmed Benali",
      "license": "PPL-12345"
    }
  }
}
```

6.2 Réponse du dashboard radar (GET /api/radar/dashboard)

Listing 12 – Dashboard radar


```
{
  "aircraft": [
    {
      "id": 45,
      "registration": "CN-AT06",
      "model": "A320",
      "status": "EN_VOL",
      "position": {
        "latitude": 33.367500,
        "longitude": -7.589800
      },
      "altitude": 35000,
      "speed": 850,
      "heading": 180
    }
  ],
  "flights": [
    {
      "id": 123,
      "flightNumber": "CMN001",
      " aircraftId ":      45 ,
      "departureAirport": "CMN",
      "arrivalAirport": "RBA",
      "flightStatus": "EN_COURS"
    }
  ],
  "weather": [
    {
      " airportId ":    1 ,
      "airportCode": "CMN",
      "windSpeed": 15,
      "windDirection": 270,
      "visibility": 10,
      "temperature": 22,
      "conditions": "Clear"
    }
  ]
}
```